A. D. Patel Institute Of Technology
(a constituent of the CVM university)

University Name: CVM University

College Name: A. D. Patel Institute of Technology

Course Title: Programming with JAVA

Course Code: 202044502

Assignment No.: Assignment -2

Student Name: Parmar Dwija

Enrollment No.: 12502080603005

Semester: 4th semester

Academic year: 2025 – 26

Program – 1

Aim: Find prime numbers using multithreading (Thread, Runnable, Executor Framework)

Objective:

To develop a Java program that finds prime numbers using multithreading techniques such as Thread class, Runnable interface, and Executor Framework to improve performance.

Source Code:

```
import java.util.concurrent.*;
// Class to check prime numbers
class PrimeTask implements Runnable {
   int number;
   // Constructor
   PrimeTask(int number) {
      this.number = number;
   }
   // Method to check prime
   boolean isPrime(int n) {
      if (n <= 1)
         return false;
      for (int i = 2; i <= n / 2; i++) {
         if (n % i == 0)
            return false;
      }
      return true;
   }

   // Run method
   public void run() {
      if (isPrime(number)) {
         System.out.println(number + " is Prime");
      }
   }
```

```
}
public class Program_1 {
   public static void main(String[] args) {
      ExecutorService executor = Executors.newFixedThreadPool(3);

      // Check numbers from 1 to 20
      for (int i = 1; i <= 20; i++) {
         executor.execute(new PrimeTask(i));
      }
      executor.shutdown();
   }
}
```

Output:

```
3 is Prime
5 is Prime
7 is Prime
11 is Prime
13 is Prime
2 is Prime
17 is Prime
19 is Prime

Process finished with exit code 0
```

Program – 2

Aim: Producer–Consumer problem using synchronization and inter-thread communication

Objective:

To implement the Producer–Consumer problem using synchronization and inter-thread communication methods like wait() and notify().

Source Code:

```
class SharedBuffer {
   int data;
   boolean valueSet = false;

   // Produce method
   synchronized void produce(int value) {
      while (valueSet) {
         try {
            wait();
         } catch (Exception e) {
         }
      }

      data = value;
      System.out.println("Produced: " + data);

      valueSet = true;
      notify();
   }

   // Consume method
   synchronized void consume() {
      while (!valueSet) {
         try {
            wait();
         } catch (Exception e) {
```

```
            }
        }

        System.out.println("Consumed: " + data);

        valueSet = false;
        notify();
    }
}

// Producer class
class Producer extends Thread {
    SharedBuffer buffer;

    Producer(SharedBuffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.produce(i);
        }
    }
}

// Consumer class
class Consumer extends Thread {
    SharedBuffer buffer;

    Consumer(SharedBuffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.consume();
```

```
        }
    }
}

public class Program_2 {
    public static void main(String[] args) {

        SharedBuffer buffer = new SharedBuffer();

        Producer p = new Producer(buffer);
        Consumer c = new Consumer(buffer);

        p.start();
        c.start();
    }
}
```

Output:

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

Process finished with exit code 0
```

Program – 3

Aim: Implement CRUD operations using Collection API (ArrayList, HashMap, TreeMap)

Objective:
To develop a Java program that performs Create, Read, Update, and Delete (CRUD) operations using Collection API classes such as ArrayList, HashMap, and TreeMap.

Source Code:

```
import java.util.*;

// Student class
class Student {
   int id;
   String name;

   // Constructor
   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
}

public class Program_3 {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      // Using ArrayList
      ArrayList<Student> list = new ArrayList<>();

      // Using HashMap
      HashMap<Integer, Student> hmap = new HashMap<>();

      // Using TreeMap
```

```
TreeMap<Integer, Student> tmap = new TreeMap<>();

int choice;

do {
   System.out.println("\n---- CRUD MENU ----");
   System.out.println("1. Add Student");
   System.out.println("2. View Students");
   System.out.println("3. Update Student");
   System.out.println("4. Delete Student");
   System.out.println("5. Exit");

   System.out.print("Enter your choice: ");
   choice = sc.nextInt();

   switch (choice) {

      // CREATE
      case 1:
         System.out.print("Enter Student ID: ");
         int id = sc.nextInt();

         System.out.print("Enter Student Name: ");
         String name = sc.next();

         Student s = new Student(id, name);

         // Add to ArrayList
         list.add(s);

         // Add to HashMap
         hmap.put(id, s);

         // Add to TreeMap
         tmap.put(id, s);
```

```
    System.out.println("Student Added Successfully");
    break;

  // READ
  case 2:
    System.out.println("\nArrayList Data:");
    for (Student st : list) {
      System.out.println(st.id + " " + st.name);
    }

    System.out.println("\nHashMap Data:");
    for (Student st : hmap.values()) {
      System.out.println(st.id + " " + st.name);
    }

    System.out.println("\nTreeMap Data (Sorted by ID):");
    for (Student st : tmap.values()) {
      System.out.println(st.id + " " + st.name);
    }
    break;

  // UPDATE
  case 3:
    System.out.print("Enter ID to Update: ");
    int uid = sc.nextInt();

    if (hmap.containsKey(uid)) {

      System.out.print("Enter New Name: ");
      String newName = sc.next();

      Student updatedStudent =
            new Student(uid, newName);

      // Update HashMap & TreeMap
      hmap.put(uid, updatedStudent);
```

```
            tmap.put(uid, updatedStudent);

            // Update ArrayList
            for (Student st : list) {
                if (st.id == uid) {
                    st.name = newName;
                }
            }

            System.out.println("Student Updated");
        } else {
            System.out.println("Student Not Found");
        }
        break;

    // DELETE
    case 4:
        System.out.print("Enter ID to Delete: ");
        int did = sc.nextInt();

        hmap.remove(did);
        tmap.remove(did);

        // Remove from ArrayList
        list.removeIf(st -> st.id == did);

        System.out.println("Student Deleted");
        break;

    case 5:
        System.out.println("Program Ended");
        break;

    default:
        System.out.println("Invalid Choice");
}
```

```
        } while (choice != 5);

        sc.close();
    }
}
```

Output:

```
---- CRUD MENU ----
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: aayushi
Student Added Successfully

---- CRUD MENU ----
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter Student ID: 102
Enter Student Name: krisha
```

```
---- CRUD MENU ----
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 2

ArrayList Data:
101 aayushi
102 krisha

HashMap Data:
101 aayushi
102 krisha

TreeMap Data (Sorted by ID):
101 aayushi
102 krisha
```

```
---- CRUD MENU ----
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 3
Enter ID to Update: 101
Enter New Name: anjali
Student Updated

---- CRUD MENU ----
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 4
Enter ID to Delete: 102
Student Deleted
```

```
---- CRUD MENU ----
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 2

ArrayList Data:
101 anjali

HashMap Data:
101 anjali

TreeMap Data (Sorted by ID):
101 anjali
```

Program -4

Aim: Sort Book objects using Comparable and Comparator interfaces

Objective:
To create a Java program that sorts Book objects using Comparable and Comparator interfaces based on different attributes like title and price.

Source Code:

```
import java.util.*;

// Book class
class Book implements Comparable<Book> {

    int id;
    String title;
    double price;

    // Constructor
    Book(int id, String title, double price) {
        this.id = id;
        this.title = title;
        this.price = price;
    }

    // Comparable: Sort by price
    public int compareTo(Book b) {

        if (this.price > b.price)
            return 1;
        else if (this.price < b.price)
            return -1;
        else
            return 0;
    }
```

```
}

// Comparator class to sort by title
class TitleComparator implements Comparator<Book> {

    public int compare(Book b1, Book b2) {
        return b1.title.compareTo(b2.title);
    }
}

public class Program_4 {
    public static void main(String[] args) {

        ArrayList<Book> list = new ArrayList<>();

        // Adding book objects
        list.add(new Book(1, "Java", 500));
        list.add(new Book(2, "Python", 400));
        list.add(new Book(3, "C++", 300));

        // Sort by price
        Collections.sort(list);

        System.out.println("Sorted by Price:");

        for (Book b : list) {
            System.out.println(b.id + " " + b.title + " " + b.price);
        }

        // Sort by title
        Collections.sort(list, new TitleComparator());

        System.out.println("\nSorted by Title:");

        for (Book b : list) {
            System.out.println(b.id + " " + b.title + " " + b.price);
```

```
        }
    }
}
```

Output:

```
Sorted by Price:
3 C++ 300.0
2 Python 400.0
1 Java 500.0

Sorted by Title:
3 C++ 300.0
1 Java 500.0
2 Python 400.0

Process finished with exit code 0
```

Program – 5

Aim: Count word occurrences from a file using File Handling APIs

Objective:
To develop a Java program that reads data from a file and counts the occurrence of each word using Java File Handling APIs.

Source Code:

```
import java.io.*;
import java.util.*;
public class Program_5 {
   public static void main(String[] args) {

      HashMap<String, Integer> map = new HashMap<>();

      try {

         BufferedReader br =
               new BufferedReader(
                     new
FileReader("C:\\Users\\SIS\\Documents\\aayushijava\\src\\Assignment_2\\aayushi.txt"));

         String line;

         while ((line = br.readLine()) != null) {

            String words[] = line.split(" ");

            for (String word : words) {

               word = word.toLowerCase();

               if (map.containsKey(word)) {
```

```
                map.put(word,
                        map.get(word) + 1);

            } else {

                map.put(word, 1);
            }
        }
    }

    br.close();

    System.out.println("Word Count:");

    for (String word : map.keySet()) {

        System.out.println(word + " : "
                + map.get(word));
    }

  } catch (Exception e) {

    System.out.println("File not found");
  }
 }
}
```

Output:

```
Word Count:
important : 1
very : 1
java : 2
is : 2
best : 1
language : 2
programming : 1

Process finished with exit code 0
```

Program – 6

Aim: Display all files of a given directory using File class

Objective:
To create a Java program that displays all files present in a specified directory using the File class.

Source Code:

```
import java.io.File;

public class Program_6
{
   public static void main(String[] args) {

      // Change this path to your folder
      File folder = new File("C:\\Users\\SIS\\Documents\\aayushijava\\src\\Assignment_1");

      // Check if folder exists
      if (folder.exists() && folder.isDirectory()) {

         File[] files = folder.listFiles();

         System.out.println("Files in Directory:");

         for (File file : files) {

            // Check only files (not folders)
            if (file.isFile()) {

               System.out.println(file.getName());
            }
         }

      } else {
```

```
        System.out.println("Directory not found or invalid path");
    }
  }
}
```

Output:

```
Files in Directory:
Program_1.java
Program_2.java
Program_3.java
Program_4.java
Program_5.java
Program_6.java
Program_7.java
Program_8.java

Process finished with exit code 0
```

Program – 7

Aim: Implement TCP Echo Client–Server program

Objective:
To develop a TCP Client–Server application in Java where the client sends a message and the server returns the same message (echo) using socket programming.

Source code:

1. Server side:

```
import java.io.*;
import java.net.*;
public class Program_7_Server
{
   public static void main(String[] args) {

      try {

         ServerSocket server =
               new ServerSocket(5000);

         System.out.println("Server Started...");

         Socket socket = server.accept();

         BufferedReader in =
               new BufferedReader(
                     new InputStreamReader(
                           socket.getInputStream()));

         PrintWriter out =
               new PrintWriter(
                     socket.getOutputStream(), true);

         String message;
```

```
        while ((message = in.readLine()) != null) {

            System.out.println("Client: " + message);

            out.println("Echo: " + message);
        }

        socket.close();
        server.close();

    } catch (Exception e) {

        System.out.println(e);
    }
  }
}
```

2. client side:

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Program_7_client {
  public static void main(String[] args) {

    try {

        Socket socket =
            new Socket("localhost", 5000);

        BufferedReader in =
            new BufferedReader(
                new InputStreamReader(
```

```
                socket.getInputStream()));

        PrintWriter out =
            new PrintWriter(
                socket.getOutputStream(), true);

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter message: ");

        String message = sc.nextLine();

        out.println(message);

        String response = in.readLine();

        System.out.println(response);

        socket.close();

    } catch (Exception e) {

        System.out.println(e);
    }
  }
}
```

Output:

```
Server Started...
Client: hello

Process finished with exit code 0
```

```
Enter message: hello
Echo: hello

Process finished with exit code 0
```

Program – 8

Aim: Develop GUI-based Investment Calculator using Swing

Objective:
To design a GUI-based Investment Calculator using Java Swing that calculates the final investment amount based on principal, rate, and time entered by the user.

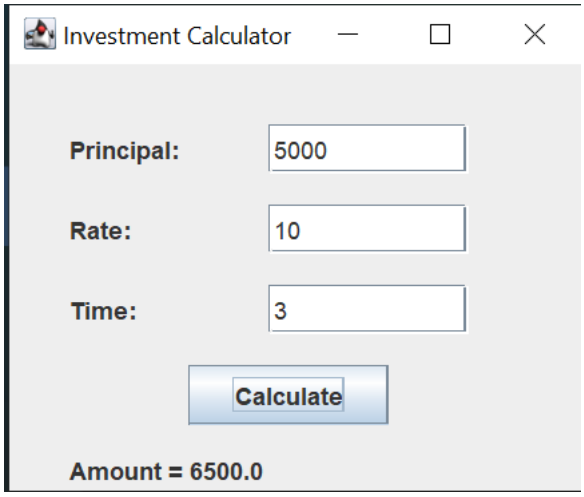
Source Code:

```
import javax.swing.*;
import java.awt.event.*;

public class Program_8 extends JFrame implements ActionListener {
   JTextField principalField;
   JTextField rateField;
   JTextField timeField;

   JLabel resultLabel;
   JButton calculateButton;

   Program_8() {

      setTitle("Investment Calculator");

      setSize(300, 250);

      setLayout(null);

      JLabel l1 = new JLabel("Principal:");
      l1.setBounds(30, 30, 100, 25);
      add(l1);

      principalField = new JTextField();
      principalField.setBounds(130, 30, 100, 25);
      add(principalField);
```

```
    JLabel l2 = new JLabel("Rate:");
    l2.setBounds(30, 70, 100, 25);
    add(l2);

    rateField = new JTextField();
    rateField.setBounds(130, 70, 100, 25);
    add(rateField);

    JLabel l3 = new JLabel("Time:");
    l3.setBounds(30, 110, 100, 25);
    add(l3);

    timeField = new JTextField();
    timeField.setBounds(130, 110, 100, 25);
    add(timeField);

    calculateButton =
          new JButton("Calculate");

    calculateButton.setBounds(90, 150, 100, 30);

    add(calculateButton);

    resultLabel = new JLabel("Result:");

    resultLabel.setBounds(30, 190, 200, 25);

    add(resultLabel);

    calculateButton.addActionListener(this);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    setVisible(true);
}
```

```
    public void actionPerformed(ActionEvent e) {

        try {

            double principal =
                Double.parseDouble(
                    principalField.getText());

            double rate =
                Double.parseDouble(
                    rateField.getText());

            double time =
                Double.parseDouble(
                    timeField.getText());

            double amount =
                principal + (principal * rate * time) / 100;

            resultLabel.setText("Amount = " + amount);

        } catch (Exception ex) {

            resultLabel.setText("Enter valid numbers");
        }
    }

    public static void main(String[] args) {

        new Program_8();
    }
}
```

Output:


Investment Calculator
Principal:
5000
Rate:
10
Time:
3
Calculate
Amount = 6500.0